1. **Create a customized Docker image using a Dockerfile.**

Objective

Build a Docker image for the frontend (HTML/CSS/JS) source code, run it as a container using Nginx, and push the image to Docker Hub

Purpose

- Containerize the static frontend for consistent and portable deployment.
- Eliminate environment dependency issues.
- Enable easy distribution via Docker Hub registry.

Environment

- OS: Amazon Linux 2023 (EC2)
- User: ec2-user
- Docker Version: 25.0.14
- Base Image: nginx:alpine
- Docker Hub Username: sunith18

Steps Performed

Step 1 — Navigate to the frontend source folder

cd ~/docker-tasks/Frontend_based_source

ls -lh

Step 2 — Create the Dockerfile

vi Dockerfile

```
FROM nginx:alpine
RUN rm -rf /usr/share/nginx/html/*
COPY . /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Step 3 — Build the Docker image

```
docker build -t frontend-app:v1 .
```

Step 4 — Run the container

docker run -d -p 8080:80 --name frontend-container frontend-app:v1

Step 5 — Verify the container

docker ps

Step 6 — Test the application

curl http://localhost:8080

Step 7 — Login to Docker Hub

docker login

Step 8 — Tag the image

docker tag frontend-app:v1 sunith18/frontend-app:v1

Step 9 — Push the image to Docker Hub

docker push sunith18/frontend-app:v1

Step 10 — Verify on Docker Hub

URL: https://hub.docker.com/r/sunith18/frontend-app

Commands Summary

Step	Command
Navigate	cd ~/docker-tasks/Frontend_based_source
Create Dockerfile	nano Dockerfile
Build image	docker build -t frontend-app:v1 .
Run container	docker run -d -p 8080:80 --name frontend-container frontend-app:v1
Verify running	docker ps
Test	curl http://localhost:8080
Login	docker login
Tag	docker tag frontend-app:v1 sunith18/frontend-app:v1
Push	docker push sunith18/frontend-app:v1

Issue Faced

403 Forbidden error from Nginx during the first attempt.

Root Cause

The Dockerfile was created in the parent directory (docker-tasks/) instead of inside Frontend_based_source/. As a result, COPY . did not copy the frontend files, leaving the Nginx web root empty.

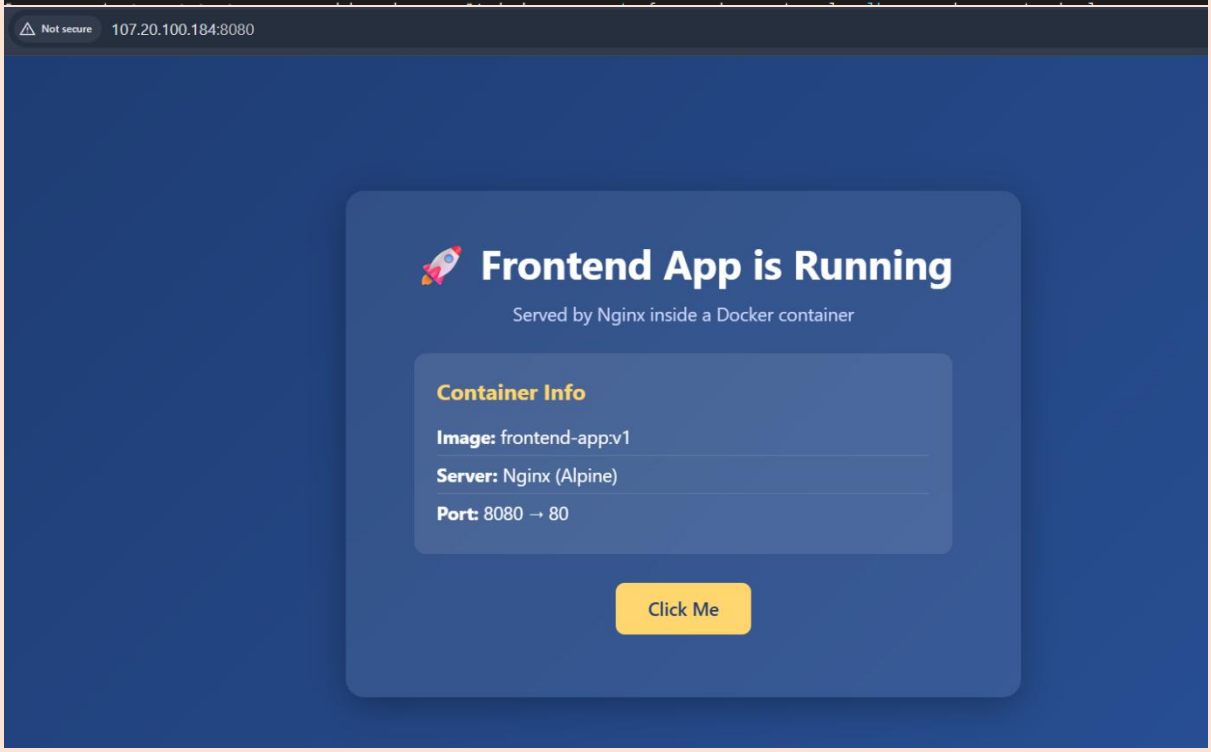
Resolution

- Moved/created the Dockerfile inside Frontend_based_source/.
- Rebuilt and re-ran the container successfully.
- curl http://localhost:8080 returned the frontend HTML.

Implementation:

```
[ec2-user@ip-172-31-19-170 Frontend_based_source]$ cd ~/docker-tasks
[ec2-user@ip-172-31-19-170 docker-tasks]$ ls
Frontend_based_source  Java_based_source  NodeJs_based_source  README.md
[ec2-user@ip-172-31-19-170 docker-tasks]$ ls -lh
total 4.0K
drwxr-xr-x. 2 ec2-user ec2-user 119 Oct 15 07:47 Frontend_based_source
drwxr-xr-x. 3 ec2-user ec2-user  32 Oct 15 07:24 Java_based_source
drwxr-xr-x. 4 ec2-user ec2-user  76 Oct 15 07:24 NodeJs_based_source
-rw-r--r--. 1 ec2-user ec2-user 488 Oct 15 07:24 README.md
[ec2-user@ip-172-31-19-170 docker-tasks]$ cd ~/docker-tasks
ls
Frontend_based_source  Java_based_source  NodeJs_based_source  README.md
[ec2-user@ip-172-31-19-170 docker-tasks]$ cd <the-frontend-folder-name-you-see>
ls -lh
-bash: syntax error near unexpected token `newline'
total 4.0K
drwxr-xr-x. 2 ec2-user ec2-user 119 Oct 15 07:47 Frontend_based_source
drwxr-xr-x. 3 ec2-user ec2-user  32 Oct 15 07:24 Java_based_source
drwxr-xr-x. 4 ec2-user ec2-user  76 Oct 15 07:24 NodeJs_based_source
-rw-r--r--. 1 ec2-user ec2-user 488 Oct 15 07:24 README.md
[ec2-user@ip-172-31-19-170 docker-tasks]$ pwd
/home/ec2-user/docker-tasks
[ec2-user@ip-172-31-19-170 docker-tasks]$ nano Dockerfile
```

```
[ec2-user@ip-172-31-19-170 Frontend_based_source]$ docker build -t frontend-app:v1 .
[+] Building 0.3s (8/8) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 226B
 => [internal] load metadata for docker.io/library/nginx:alpine
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/3] FROM docker.io/library/nginx:alpine@sha256:72aeb3308ef6cb3118242fb16389518923916dfa206e32d1a8377acc328b9870
 => [internal] load build context
 => => transferring context: 19.32kB
 => CACHED [2/3] RUN rm -rf /usr/share/nginx/html/*
 => [3/3] COPY . /usr/share/nginx/html/
 => exporting to image
 => => exporting layers
 => => writing image sha256:7830ea1b2492c142243164b987999e7a661a9cf06b5f423989f0a260e7f3c896
 => => naming to docker.io/library/frontend-app:v1
[ec2-user@ip-172-31-19-170 Frontend_based_source]$ docker run -d -p 8080:80 --name frontend-container frontend-app:v1
1ced2accb8e894cfe13b2f6f09402e50542ada2af00174587bad8437cb825156
```



Conclusion

The frontend application was successfully containerized using an Nginx base image, tested locally, and pushed to Docker Hub. The task demonstrated how static frontend applications can be packaged and distributed using Docker, ensuring portability and consistency across environments.

2. Push the image to Docker Hub.

Objective

Push the locally built Docker image (frontend-app:v1) to Docker Hub so it can be pulled and deployed from any environment

Purpose

- Store the image in a central, public/private registry.
- Enable version control of Docker images using tags.
- Allow other systems (CI/CD, EC2, ECS, Kubernetes) to pull the image easily.

Environment

- **OS:** Amazon Linux 2023 (EC2)
- **User:** ec2-user
- **Docker Version:** 25.0.14

- **Docker Hub Username:** sunith18
- **Image Name:** frontend-app
- **Tag:** v1

Prerequisites

- Docker installed and running on EC2.
- Image frontend-app:v1 already built locally.
- Docker Hub account created.
- Internet access from EC2

Steps Performed

Step 1 — Verify the local image exists

docker images | grep frontend-app

Step 2 — Login to Docker Hub

docker login

Entered:

- Username: sunith18
- Password: Docker Hub password / Personal Access Token

Step 3 — Tag the image with Docker Hub repository name

docker tag frontend-app:v1 sunith18/frontend-app:v1

Explanation:

- frontend-app:v1 → local image name and tag
- sunith18/frontend-app:v1 → Docker Hub repository path with tag

Step 4 — Push the image to Docker Hub

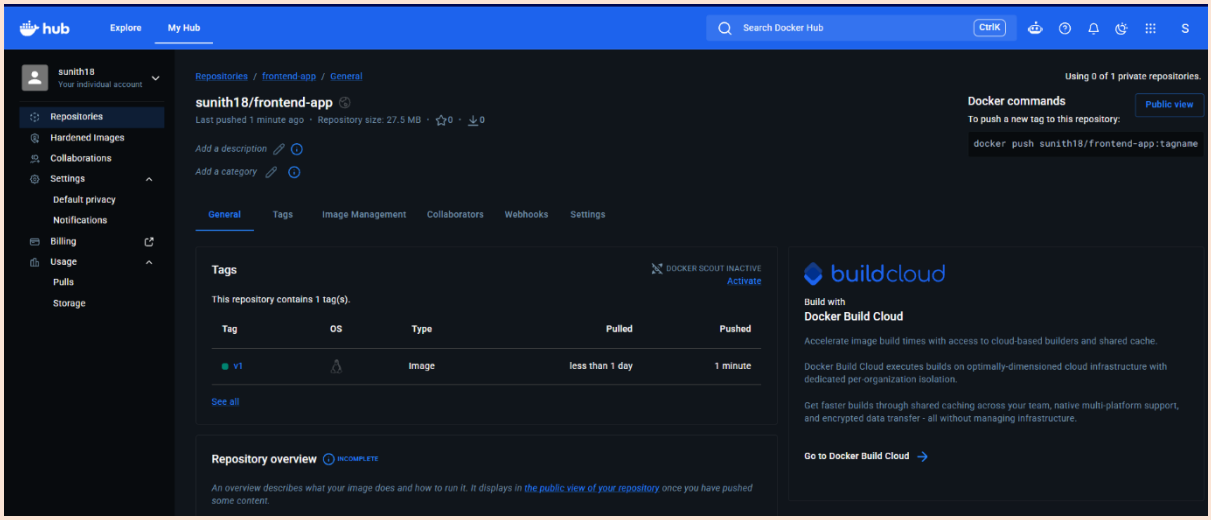
docker push sunith18/frontend-app:v1

Step 5 — Verify the push

- Docker Hub URL: https://hub.docker.com/r/sunith18/frontend-app
- The v1 tag is listed with the correct image size.

Implementation:

```
[ec2-user@ip-172-31-19-170 Frontend_based_source]$ docker tag frontend-app:v1 sunith18/frontend-app:v1
docker push sunith18/frontend-app:v1
The push refers to repository [docker.io/sunith18/frontend-app]
8f95c8769508: Pushed
9575dc0b1a34: Pushed
6aef7713033e: Mounted from library/nginx
a5237c8bfc43: Mounted from library/nginx
1cbec789df91: Mounted from library/nginx
c01de5b74a88: Mounted from library/nginx
f4b56df1052e: Mounted from library/nginx
11c068032a92: Mounted from library/nginx
1d5f8e4b80b9: Mounted from library/nginx
74d97c428c51: Mounted from library/nginx
v1: digest: sha256:98d156a42f98dc6cc070fc2e49f3770aee3accffafe3e272a7806c145894f13f size: 2404
[ec2-user@ip-172-31-19-170 Frontend_based_source]$ docker images | grep sunith18
sunith18/frontend-app   v1        7830ea1b2492   4 minutes ago   69.1MB
```



Conclusion

The customized Docker image was successfully pushed to Docker Hub. This confirms that the image is now available in a public registry and can be pulled from any machine or CI/CD pipeline using: docker pull sunith18/frontend-app:v1

3. Push the same image to Amazon ECR.

Objective

Push the existing Docker image (frontend-app:v1) to Amazon Elastic Container Registry (ECR) in the AWS account.

Purpose

- Store Docker images in a private, secure AWS-managed registry.
- Enable AWS services (ECS, EKS, EC2) to pull the image directly.
- Provide version control and image scanning capabilities.
- Avoid dependency on external registries like Docker Hub for production workloads.

Environment

- **AWS Account ID:** 567487919764
- **Region:** us-east-1
- **ECR Repository:** frontend-app
- **Repository URI:** 567487919764.dkr.ecr.us-east-1.amazonaws.com/frontend-app
- **Image Tag:** v1
- **Source Image:** frontend-app:v1 (built in Task 1)
- **EC2 IAM Role:** EC2-CloudWatchAgent-Role (with inline policy EC2-ECR-Frontend-App-Policy)

Prerequisites

- EC2 instance with Docker installed.
- AWS CLI installed (awscli 2.36.47).
- IAM role attached to EC2 with ECR permissions:
  - ecr:GetAuthorizationToken
  - ecr:CreateRepository
  - ecr:DescribeRepositories
  - ecr:BatchCheckLayerAvailability
  - ecr:InitiateLayerUpload

    - ecr:UploadLayerPart
    - ecr:CompleteLayerUpload
    - ecr:PutImage
    - ecr:ListImages
- Internet access from EC2 to AWS ECR endpoints.

Steps Performed

Step 1 — Verify AWS identity from EC2

```
aws sts get-caller-identity
```

Output:


```
{
  "UserId": "AROAYIIHKO2KHUIKZ2UGD:i-04909264fa1ac8f9b",
  "Account": "567487919764",
  "Arn": "arn:aws:sts::567487919764:assumed-role/EC2-CloudWatchAgent-Role/i-04909264fa1ac8f9b"
}
```


Step 2 — Create the ECR repository

```
aws ecr create-repository --repository-name frontend-app --region us-east-1
```

Output:


```
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:567487919764:repository/frontend-app",
    "registryId": "567487919764",
    "repositoryName": "frontend-app",
    "repositoryUri": "567487919764.dkr.ecr.us-east-1.amazonaws.com/frontend-app",
    "createdAt": "2026-10-15T09:15:14.222000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": { "scanOnPush": false },
    "encryptionConfiguration": { "encryptionType": "AES256" }
  }
}
```


Step 3 — Authenticate Docker to ECR

```
AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
AWS_REGION=us-east-1
aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com
```

Result: Login Succeeded

Step 4 — Tag the existing local image for ECR

```
docker tag frontend-app:v1 $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/frontend-app:v1
```

Step 5 — Push the image to ECR

docker push $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/frontend-app:v1

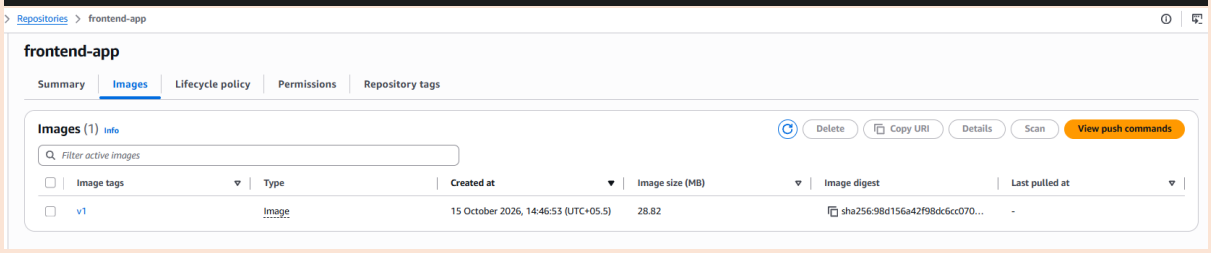
Step 6 — Verify the image in ECR

aws ecr list-images --repository-name frontend-app --region us-east-1

Implementation:

```
[ec2-user@ip-172-31-19-170 ~]$ AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
AWS_REGION=us-east-1
aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[ec2-user@ip-172-31-19-170 ~]$ docker tag frontend-app:v1 $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/frontend-app:v1
[ec2-user@ip-172-31-19-170 ~]$ docker images | grep frontend-app
567487919764.dkr.ecr.us-east-1.amazonaws.com/frontend-app   v1        7830ea1b2492   About an hour ago   69.1MB
frontend-app                                               v1        7830ea1b2492   About an hour ago   69.1MB
sunith18/frontend-app                                      v1        7830ea1b2492   About an hour ago   69.1MB
[ec2-user@ip-172-31-19-170 ~]$ docker push $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/frontend-app:v1
The push refers to repository [567487919764.dkr.ecr.us-east-1.amazonaws.com/frontend-app]
8f95c8769508: Pushed
9575dc0b1a34: Pushed
6aef7713033e: Pushed
a5237c8bfc43: Pushed
1cbec789df91: Pushed
c01de5b74a88: Pushed
f4b56df1052e: Pushed
11c068032a92: Pushed
1d5f8e4b80b9: Pushed
74d97c428c51: Pushed
v1: digest: sha256:98d156a42f98dc6cc070fc2e49f3770aee3accffafe3e272a7806c145894f13f size: 2404
[ec2-user@ip-172-31-19-170 ~]$
```



Conclusion

The Docker image frontend-app:v1 was successfully pushed to Amazon ECR. This confirms the image is now stored in AWS's private registry and is ready to be pulled by AWS services such as ECS, EKS, or EC2 instances with appropriate IAM permissions

Pull command for future use:

docker pull 567487919764.dkr.ecr.us-east-1.amazonaws.com/frontend-app:v1

4. Provision one EC2 instance using Terraform and install Jenkins.

Objective

Use Terraform (Infrastructure as Code) to provision an EC2 instance on AWS and automatically install Jenkins on it.

Purpose

- Automate infrastructure provisioning without manual console operations.
- Demonstrate Infrastructure as Code (IaC) using Terraform.
- Set up a Jenkins CI/CD server ready for pipeline jobs.
- Ensure repeatable and version-controlled infrastructure.

Environment

- **AWS Account ID:** 567487919764
- **Region:** us-east-1
- **Terraform Version:** v1.x
- **AMI:** Ubuntu 22.04 LTS (ami-062944a84867f2386)

- **Instance Type:** t3.micro (Free Tier eligible)
- **Jenkins Public IP:** 98.93.198.74
- **Jenkins Port:** 8080
- **IAM Role (Terraform host):** EC2CustomNginxECRRole

Prerequisites

- Existing EC2 with AWS CLI configured.
- IAM role with permissions: ec2:* (create/describe/run/terminate), ec2:Describe*.
- SSH key pair available.
- Terraform installed.
- Security group allowing ports 22, 80, 8080.

Architecture

```
Existing EC2 (Amazon Linux 2023)     AWS Cloud (us-east-1)
  |                                |
  |-- Terraform ------------------>  |-- Key Pair: jenkins-key
  |                                |-- Security Group: jenkins-sg (22, 80, 8080)
  |                                |-- EC2: Jenkins-Server (t3.micro, Ubuntu 22.04)
  |                                    |
  |                                    |-- Java 21
  |                                    |-- Docker
  |                                    |-- Jenkins (port 8080)
  |
  |-- Browser --> http://98.93.198.74:8080
```

Steps Performed

Step 1 — Install Terraform (Amazon Linux 2023)

```
sudo dnf install -y yum-utils

sudo yum-config-manager --add-repo
https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo

sudo dnf install -y terraform

terraform -version
```

Step 2 — Create project folder

```
mkdir -p ~/jenkins-terraform && cd ~/jenkins-terraform
```

Step 3 — Generate SSH key pair

```
ssh-keygen -t rsa -b 2048 -f ~/.ssh/jenkins_key -N ""
```

Step 4 — Fetch latest Ubuntu 22.04 AMI

```
aws ec2 describe-images \
  --owners 099720109477 \
  --filters "Name=name,Values=ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*" \
  --query 'Images|sort_by(@,&CreationDate)[-1].ImageId' \
  --output text --region us-east-1
```

Result: ami-062944a84867f2386

Step 5 — Create main.tf

```
provider "aws" { region = "us-east-1" }

resource "aws_key_pair" "jenkins_key" {
  key_name   = "jenkins-key"
  public_key = file("~/.ssh/jenkins_key.pub")
}

resource "aws_security_group" "jenkins_sg" {
  name = "jenkins-sg"
  ingress { from_port = 22   to_port = 22   protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { from_port = 8080 to_port = 8080 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { from_port = 80   to_port = 80   protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  egress  { from_port = 0    to_port = 0    protocol = "-1"  cidr_blocks = ["0.0.0.0/0"] }
}

resource "aws_instance" "jenkins" {
  ami                  = "ami-062944a84867f2386"
  instance_type        = "t3.micro"
  key_name             = aws_key_pair.jenkins_key.key_name
  vpc_security_group_ids = [aws_security_group.jenkins_sg.id]
  user_data            = file("user_data.sh")
  tags = { Name = "Jenkins-Server" }
}

output "jenkins_public_ip" { value = aws_instance.jenkins.public_ip }
```

Step 6 — Create user_data.sh

```
#!/bin/bash
set -e
apt-get update -y
apt-get install -y openjdk-21-jdk docker.io unzip curl gnupg
systemctl enable --now docker
usermod -aG docker ubuntu

curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key | \
  tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable
binary/" \
  | tee /etc/apt/sources.list.d/jenkins.list > /dev/null

apt-get update -y
apt-get install -y jenkins
systemctl enable --now Jenkins
```

Step 7 — Deploy infrastructure
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply -auto-approve

Step 8 — Fix Docker group for Jenkins
ssh -i ~/.ssh/jenkins_key ubuntu@98.93.198.74
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
exit

Step 9 — Get Jenkins initial admin password
ssh -i ~/.ssh/jenkins_key ubuntu@98.93.198.74 "sudo cat /var/lib/jenkins/secrets/initialAdminPassword"

Issues Faced and Resolutions

Issue 1 — UnauthorizedOperation: ec2:RevokeSecurityGroupEgress

- Cause: IAM role missing EC2 permissions.
- Resolution: Added inline policy EC2-Terraform-FullAccess-Policy to EC2CustomNginxECRRole.

Issue 2 — InvalidParameterCombination: t3.medium not Free Tier eligible

- Cause: t3.medium is not Free Tier.
- Resolution: Changed instance_type to t3.micro in main.tf.

Issue 3 — UnauthorizedOperation: ec2:DescribeInstanceTypes

- Cause: Policy didn't include describe permissions.
- Resolution: Added ec2:Describe* to the IAM policy.

Issue 4 — Terraform state out of sync after instance creation failed mid-apply

- Cause: Instance was created in AWS but Terraform didn't record it.
- Resolution: terraform import aws_instance.jenkins <instance-id> then re-applied. Terraform replaced it and recorded cleanly.

Issue 5 — Jenkins GPG key error (NO_PUBKEY 7198F4B714ABFC68**)**

- Cause: Jenkins rotated its signing key in 2026; old jenkins.io-2023.key URL expired.
- Resolution: Switched to jenkins.io-2026.key and re-added repo.

Issue 6 — Jenkins service failed: "Start request repeated too quickly"

- Cause: Jenkins 2.580.1 requires Java 21, but only Java 17 was installed.
- Resolution: Installed openjdk-21-jdk, cleared failed state (systemctl reset-failed jenkins), restarted Jenkins.

Implementation:

```
ami-062944a84867f2386
[ec2-user@ip-172-31-19-170 jenkins-terraform]$ vi main.tf
[ec2-user@ip-172-31-19-170 jenkins-terraform]$ nano user_data.sh
[ec2-user@ip-172-31-19-170 jenkins-terraform]$ terraform init
terraform fmt
terraform validate
terraform plan
terraform apply -auto-approve
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.69.0...
- Installed hashicorp/aws v6.69.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
Success! The configuration is valid.
```

```
          + subnet_id                            = (known after apply)
          + tags                                 = (known after apply)
          + tags_all                             = (known after apply)
          + tenancy                              = (known after apply)
          + user_data                            = (known after apply)
          + user_data_base64                     = (known after apply)
          + user_data_replace_on_change          = (known after apply)
          + volume_tags                          = (known after apply)
          + vpc_security_group_ids               = (known after apply)
        } -> (known after apply)
    }

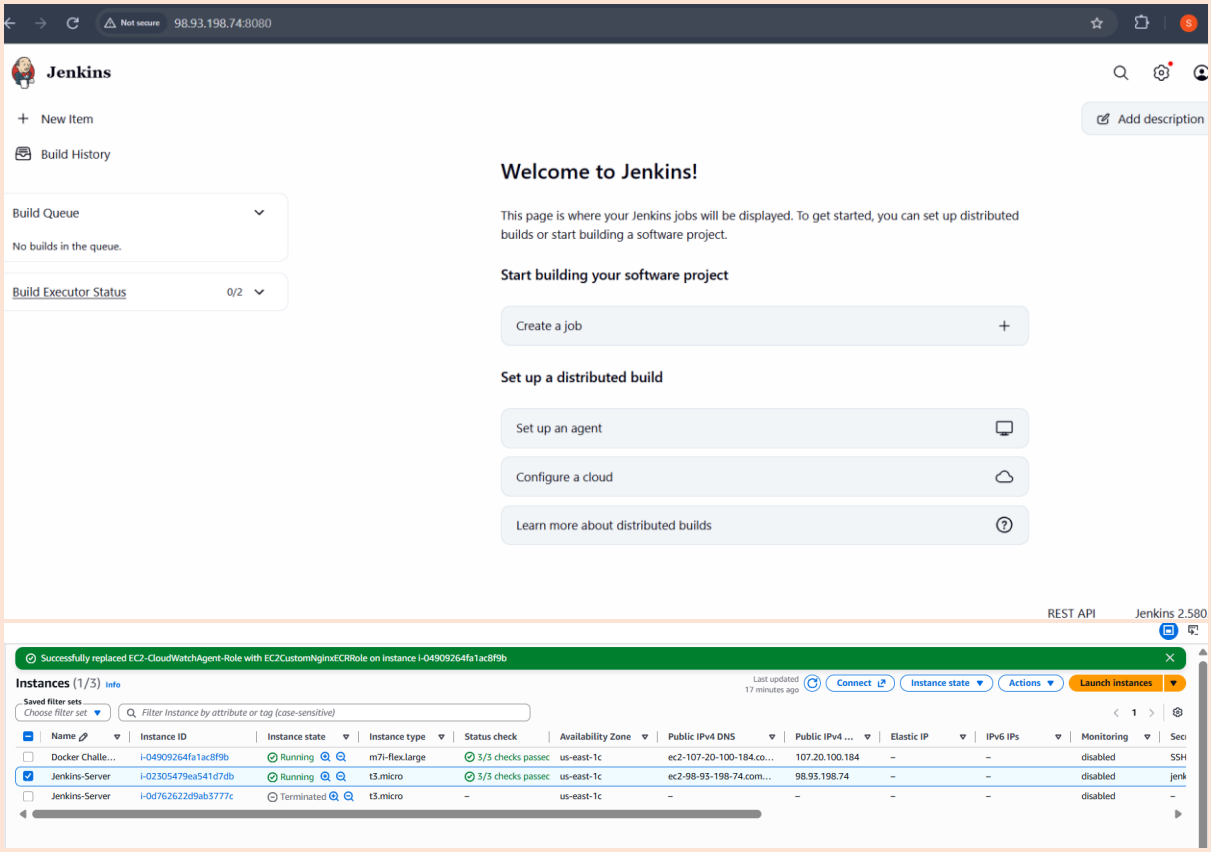
Plan: 1 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  + jenkins_public_ip = (known after apply)
aws_instance.jenkins: Destroying... [id=i-0d762622d9ab3777c]
aws_instance.jenkins: Still destroying... [id=i-0d762622d9ab3777c, 10s elapsed]
aws_instance.jenkins: Still destroying... [id=i-0d762622d9ab3777c, 20s elapsed]
aws_instance.jenkins: Destruction complete after 30s
aws_instance.jenkins: Creating...
aws_instance.jenkins: Still creating... [10s elapsed]
aws_instance.jenkins: Creation complete after 13s [id=i-02305479ea541d7db]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

jenkins_public_ip = "98.93.198.74"
[ec2-user@ip-172-31-19-170 jenkins-terraform]$
```



Conclusion

Terraform successfully provisioned an EC2 instance on AWS and installed Jenkins automatically using user_data. The Jenkins server is now accessible at http://98.93.198.74:8080 and ready for CI/CD pipelines. This task demonstrated Infrastructure as Code (IaC) principles, eliminating manual provisioning and enabling repeatable Jenkins deployments.

5. Create one Jenkins job to build and push the Docker image to Docker Hub.

- Source: https://github.com/betawins/Python-app.git

Source Codes: https://github.com/betawins/docker-tasks.git

- From the **frontend source code**, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.

Objective

Create a Jenkins Pipeline job that clones the Python app from GitHub, builds a Docker image, and pushes it to Docker Hub automatically.

Purpose

- Automate the build → push workflow using Jenkins Pipeline.
- Demonstrate CI/CD as code using a declarative Jenkinsfile.
- Enable repeatable, version-tagged image delivery.
- Establish the foundation for end-to-end CI/CD with AWS.

Docker-Challenge

Environment

- **Jenkins URL:** http://98.93.198.74:8080
- **Jenkins Version:** 2.580.1
- **Jenkins User:** admin
- **Jenkins EC2:** t3.micro Ubuntu 22.04
- **Docker Hub User:** sunith18
- **Repository:** https://github.com/betawins/Python-app.git
- **Image:** sunith18/python-app:v1
- **Source Commit:** a9c18a887f134751f06bab9eb128a4387b42c141 (branch: main)

Prerequisites

- Jenkins running and accessible (from Task 4).
- Docker installed on Jenkins EC2.
- jenkins user added to docker group.
- Jenkins plugins installed:
  - Docker Pipeline
  - Git
  - Credentials Binding
- Docker Hub credentials added in Jenkins:
  - ID: dockerhub-creds
  - Username: sunith18
  - Password: Docker Hub password / PAT

Architecture

```
GitHub (betawins/Python-app)
      |
      | git clone
      v
Jenkins EC2 (98.93.198.74)
   |
   |-- Pipeline: python-app-build
   |      |
   |      |-- Stage 1: Clone Code
   |      |-- Stage 2: Override Dockerfile (python 3.11)
   |      |-- Stage 3: Build Docker Image
   |      |-- Stage 4: Test Image
   |      |-- Stage 5: Login to Docker Hub
   |      |-- Stage 6: Push to Docker Hub
   |
   v
Docker Hub: sunith18/python-app:v1
```

Docker-Challenge

Steps Performed

Step 1 — Install required Jenkins plugins

Manage Jenkins → Plugins → Installed: Docker Pipeline, Git, Credentials Binding.

Step 2 — Add Docker Hub credentials

Manage Jenkins → Credentials → System → Global → Add Credentials:

- Kind: Username with password
- Username: sunith18
- Password: Docker Hub password
- ID: dockerhub-creds

Step 3 — Grant jenkins **Docker access**

ssh -i ~/.ssh/jenkins_key ubuntu@98.93.198.74
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
sudo -u jenkins docker ps # verified: no permission error
exit

Step 4 — Create Jenkins Pipeline job

- New Item → Pipeline → name: python-app-build
- Definition: Pipeline script
- Pasted the declarative Jenkinsfile (below)

Step 5 — Jenkinsfile used

```
pipeline {
  agent any

  environment {
    DOCKERHUB_CREDENTIALS = credentials('dockerhub-creds')
    IMAGE_NAME = "sunith18/python-app"
    IMAGE_TAG  = "v1"
  }

  stages {
    stage('Clone Code') {
      steps {
        git branch: 'main',
          url: 'https://github.com/betawins/Python-app.git'
      }
    }

    stage('Override Dockerfile (python 3.11)') {
      steps {
        sh '''
        cat > Dockerfile <<'EOF'
FROM python:3.11-slim
WORKDIR /opt
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8080
ENTRYPOINT ["python", "app.py"]
EOF
            '''
            sh 'cat Dockerfile'
          }
        }

        stage('Build Docker Image') {
          steps {
            sh 'docker build -t $IMAGE_NAME:$IMAGE_TAG .'
          }
        }

        stage('Test Image') {
          steps {
            sh 'docker images | grep $IMAGE_NAME'
          }
        }

        stage('Login to Docker Hub') {
          steps {
            sh 'echo $DOCKERHUB_CREDENTIALS_PSW | docker login -u
$DOCKERHUB_CREDENTIALS_USR --password-stdin'
          }
        }

        stage('Push to Docker Hub') {
          steps {
            sh 'docker push $IMAGE_NAME:$IMAGE_TAG'
          }
        }
    }

    post {
      always  { sh 'docker logout || true' }
      success { echo "Image $IMAGE_NAME:$IMAGE_TAG pushed to Docker Hub." }
      failure { echo "Pipeline failed. Check console output." }
    }
}
```

Step 6 — Run the build

- Clicked **Build Now**
- Watched Console Output
- All stages passed → Finished: SUCCESS

Docker-Challenge

Issues Faced and Resolutions

Issue 1 — Repo's Jenkinsfile referenced vatsraj/pythonapp

- Cause: Original repo owner's Docker Hub username.
- Resolution: Created own inline pipeline script with sunith18/python-app.

Issue 2 — Repo's Jenkinsfile referenced s3cloudhub-dockerhub **credentials ID**

- Cause: Different credentials ID from the repo author.
- Resolution: Used dockerhub-creds (matching Jenkins credentials we created).

Issue 3 — Repo's Jenkinsfile called slackSend

- Cause: Slack plugin not installed, would fail the build.
- Resolution: Removed Slack steps in our custom pipeline.

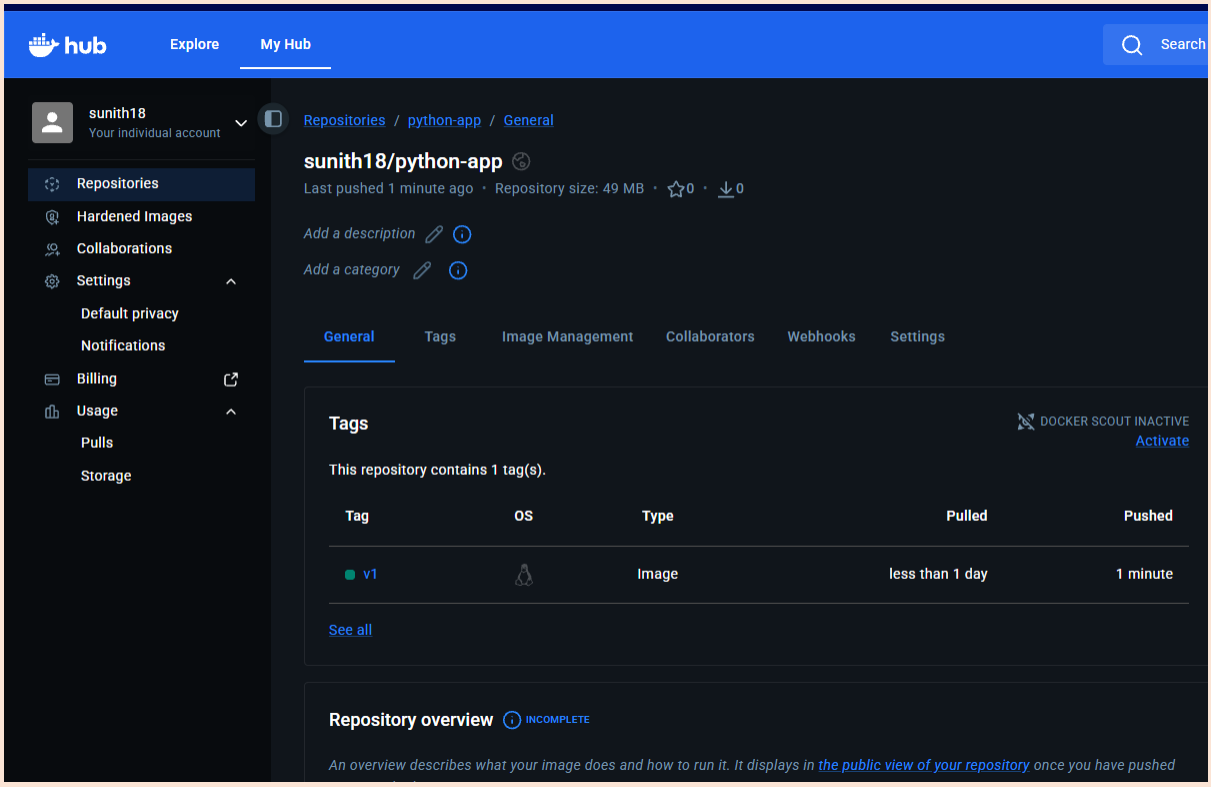
Issue 4 — Repo's Dockerfile used python:3.6

- Cause: EOL Python version; pip TLS errors common.
- Resolution: Pipeline overrides Dockerfile at build time with python:3.11-slim.

Implementation:

```
[ec2-user@ip-172-31-19-170 ~]$ docker pull sunith18/python-app:v1
v1: Pulling from sunith18/python-app
ecc510c1e359: Pull complete
0f752a1982d6: Pull complete
759a7f30c318: Pull complete
496c1a902b3a: Pull complete
6325262a0f79: Pull complete
96029530a18d: Pull complete
8f21aa03d8fa: Pull complete
Digest: sha256:f4ee5c8e9c3f56a1966cf34e764345f6275bcf42edcb5ef3c5d3455469d1fa2d
Status: Downloaded newer image for sunith18/python-app:v1
docker.io/sunith18/python-app:v1
[ec2-user@ip-172-31-19-170 ~]$
```

```
ecc510c1e359: Waiting
0f752a1982d6: Waiting
0f752a1982d6: Mounted from library/python
496c1a902b3a: Mounted from library/python
759a7f30c318: Mounted from library/python
ecc510c1e359: Mounted from library/python
8f21aa03d8fa: Pushed
6325262a0f79: Pushed
96029530a18d: Pushed
v1: digest: sha256:f4ee5c8e9c3f56a1966cf34e764345f6275bcf42edcb5ef3c5d3455469d1fa2d size: 1843
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
+ docker logout
Removing login credentials for https://index.docker.io/v1/
[Pipeline] echo
✅ Image sunith18/python-app:v1 pushed to Docker Hub.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Conclusion

A Jenkins Pipeline was successfully created to automate the build and push of the Python app Docker image to Docker Hub. The pipeline clones the repository, overrides the outdated Dockerfile with a modern Python 3.11 base, builds the image, authenticates with Docker Hub using Jenkins credentials, and pushes the tagged image. This demonstrates a working CI/CD workflow that can be extended with automated testing, ECR pushes, and deployment stages.

- From the **Java-based source code**, write a Dockerfile, build, run, and push the image to the Docker registry.

Objective: Write a Dockerfile for the Java-based source code, build the Docker image, run it as a container, verify via browser, and push the image to Docker Hub.

Purpose: Package the Java (Maven) application into a portable container for consistent deployment and share it via Docker Hub.

Prerequisites:

- Docker installed on EC2 (Amazon Linux 2023).
- Source folder: ~/docker-tasks/Java_based_source.
- pom.xml present (Maven project with src/).
- Docker Hub account: sunith18.

Steps:

1.Navigate to the source folder:

cd ~/docker-tasks/Java_based_source
ls -lh

Files: Dockerfile, pom.xml, src/

2.Inspect the Maven project:
cat pom.xml
Confirms artifact packaging (jar/war), Java version, dependencies.

3.Build the Docker image (Maven multi-stage):
docker build -t java-app:v1 .

4.Run the container:
docker run -d -p 8081:8080 --name java-container java-app:v1

5.Verify container status:
docker ps
docker logs java-container

6.Test the application:
curl http://localhost:8081

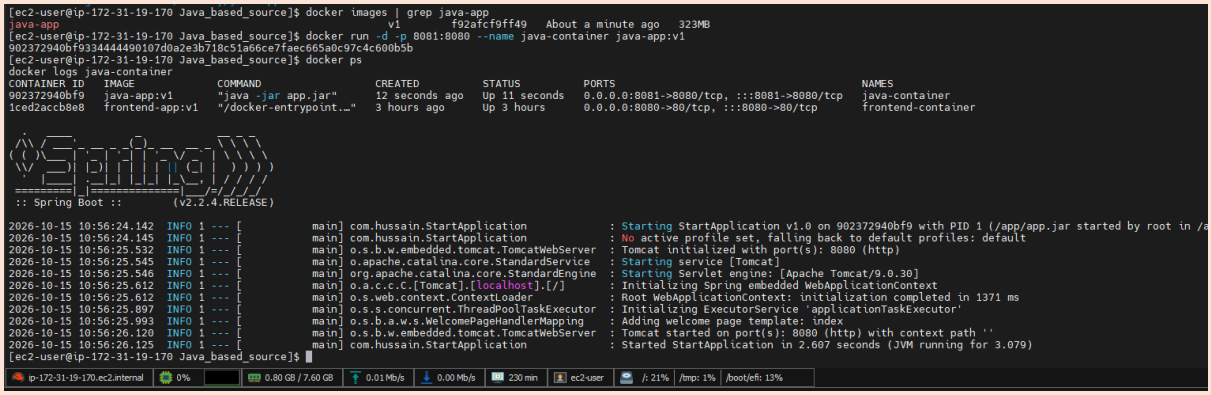
7. Tag and push to Docker Hub:
docker login
docker tag java-app:v1 sunith18/java-app:v1
docker push sunith18/java-app:v1

8. Verify on Docker Hub: https://hub.docker.com/r/sunith18/java-app
Dockerfile Used:

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests

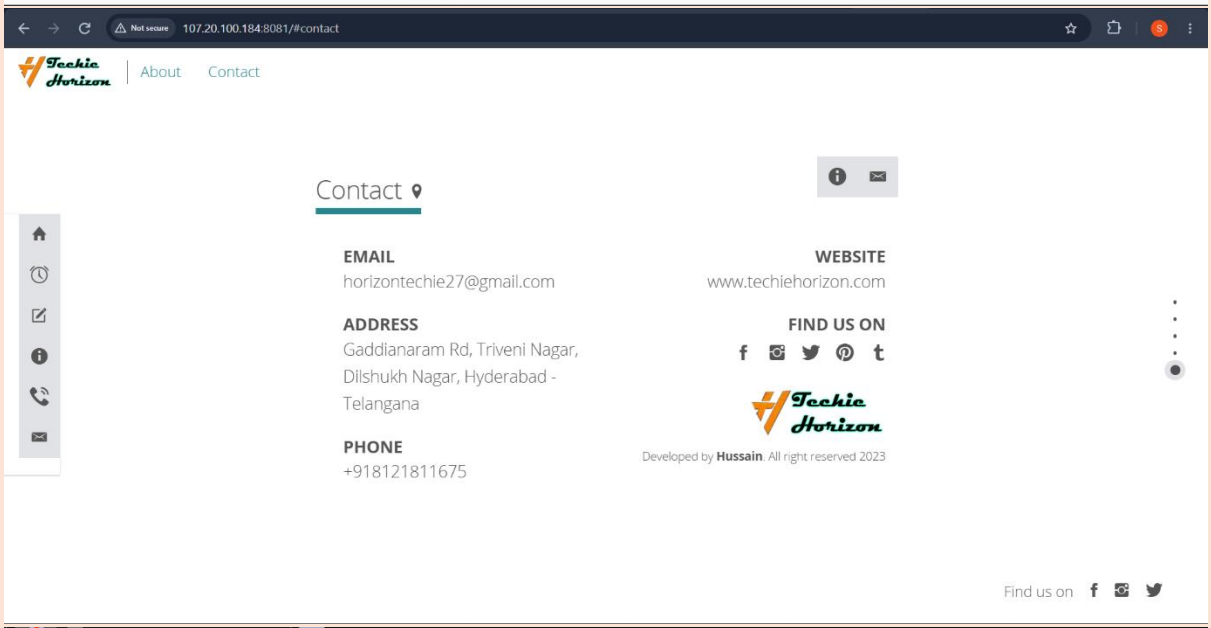
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Implementation:

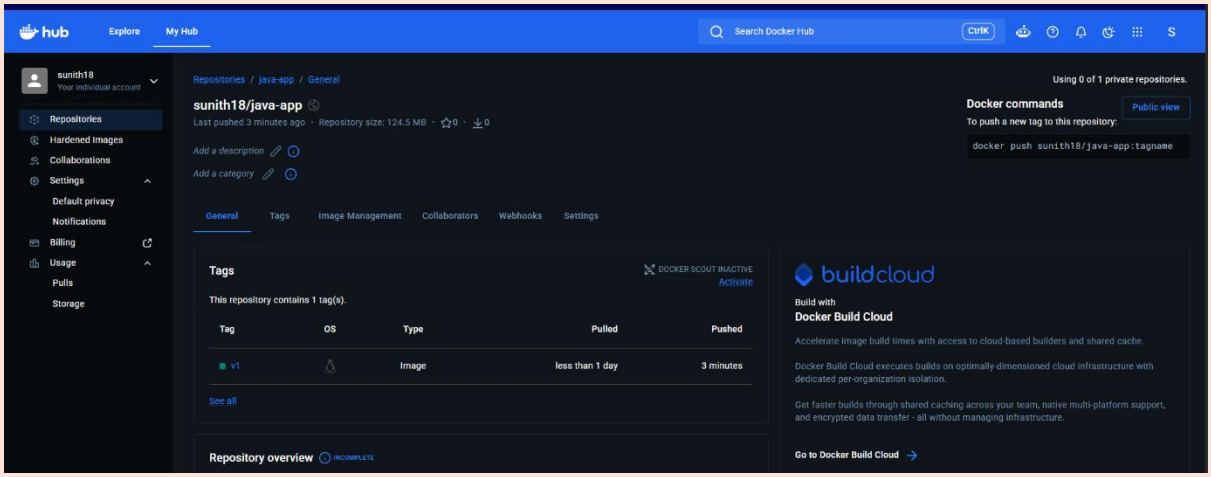


Docker-Challenge






```
[ec2-user@ip-172-31-19-170 Java_based_source]$ docker images | grep java-app
docker push sunith18/java-app:v1
java-app                                        v1        f92afcf9ff49   12 minutes ago   323MB
The push refers to repository [docker.io/sunith18/java-app]
An image does not exist locally with the tag: sunith18/java-app
[ec2-user@ip-172-31-19-170 Java_based_source]$ docker tag java-app:v1 sunith18/java-app:v1
docker push sunith18/java-app:v1
The push refers to repository [docker.io/sunith18/java-app]
ddb8cd91af3e: Pushed
a21176e664eb: Pushed
7de77111c509: Mounted from library/eclipse-temurin
b9796b1f52d1: Mounted from library/eclipse-temurin
3bc295b94679: Mounted from library/eclipse-temurin
012f3c474d69: Mounted from library/eclipse-temurin
f98d83061adc: Mounted from library/eclipse-temurin
f6e0e88dd8e1: Mounted from library/eclipse-temurin
v1: digest: sha256:4c0555a56a286e596d21d2e1fec0c1b0cb981c695b7b5bd9cca3255fb7c2f33b size: 1993
[ec2-user@ip-172-31-19-170 Java_based_source]$ docker ps
cat ~/docker-tasks/Java_based_source/pom.xml | grep -E "artifactId|packaging|java.version|maven.compiler"
CONTAINER ID   IMAGE            COMMAND                  CREATED          STATUS          PORTS                                       NAMES
902372940bf9   java-app:v1      "java -jar app.jar"      11 minutes ago   Up 11 minutes   0.0.0.0:8081->8080/tcp, :::8081->8080/tcp   java-container
1ced2accb8e8   frontend-app:v1  "/docker-entrypoint.…"   3 hours ago      Up 3 hours      0.0.0.0:8080->80/tcp, :::8080->80/tcp       frontend-container
    <artifactId>spring-boot-demo</artifactId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <maven.compiler.source>11</maven.compiler.source>
        <maven.compiler.target>11</maven.compiler.target>
        <java.version>11</java.version>
            <artifactId>spring-boot-starter-web</artifactId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
            <artifactId>spring-boot-starter-test</artifactId>
            <artifactId>spring-boot-devtools</artifactId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <artifactId>maven-compiler-plugin</artifactId>
                    <source>${java.version}</source>
                    <target>${java.version}</target>
[ec2-user@ip-172-31-19-170 Java_based_source]$
```



Conclusion:
The Java (Maven) application was successfully containerized using a multi-stage Dockerfile (Maven build stage + JRE runtime stage), tested locally, and published to Docker Hub for reuse.

- From the **Node.js-based source code**, write a Dockerfile, build with tag v1, run, and push it to the Docker registry.

Objective:
Write a Dockerfile for the Node.js-based source code, build the image with tag v1, run it as a container, and push it to Docker Hub.

Purpose:
Package the Node.js application into a portable container for consistent deployment and share it via Docker Hub.

Prerequisites:

- Docker installed on EC2.
- Source folder: ~/docker-tasks/NodeJs_based_source.
- package.json present.
- Docker Hub account: sunith18.

Steps:
1.Navigate to the source folder:

```
cd ~/docker-tasks/NodeJs_based_source
ls -R
cat package.json
```

2.Create the **Dockerfile** with **node:20-alpine** base.
3.Build the image with tag v1:

```
docker build -t node-app:v1 .
```

4.Run the container:

```
docker run -d -p 3000:3000 --name node-container node-app:v1
```

5.Verify container status:

docker ps

docker logs node-container

6.Test the application:

curl http://localhost:3000

Also verified in browser at http://<EC2_PUBLIC_IP>:3000.

7.Login, tag, and push:

docker login

docker tag node-app:v1 sunith18/node-app:v1

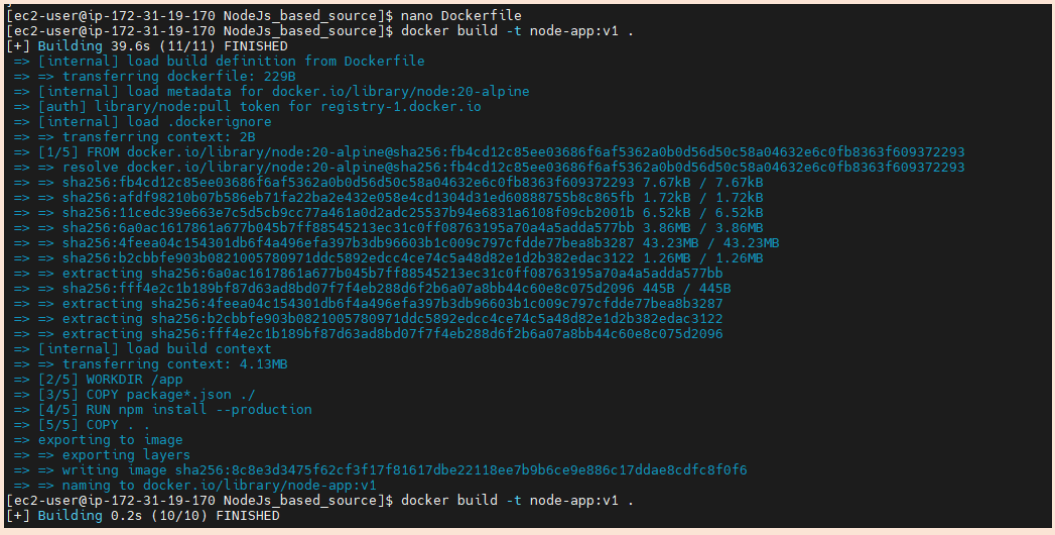
docker push sunith18/node-app:v1

8. Verify on Docker Hub: https://hub.docker.com/r/sunith18/node-app → tag v1 present.

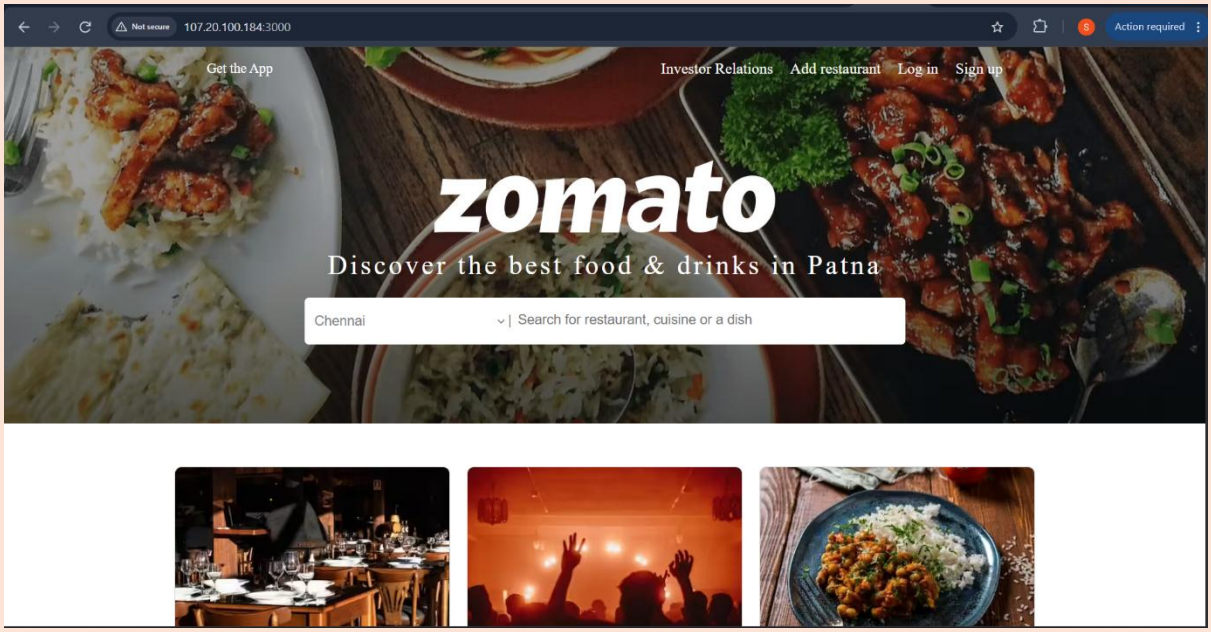
Dockerfile Used:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Implementation:



```
[ec2-user@ip-172-31-19-170 NodeJs_based_source]$ docker tag node-app:v1 sunith18/node-app:v1
[ec2-user@ip-172-31-19-170 NodeJs_based_source]$ docker push sunith18/node-app:v1
The push refers to repository [docker.io/sunith18/node-app]
fe2447a91b6f: Pushed
391f85b9f3e5: Pushed
38a682d904c8: Pushed
c967a68561bb: Pushed
afa543f85b46: Mounted from library/node
e10358715ead: Mounted from library/node
4983b93ee796: Mounted from library/node
29df493baa13: Mounted from library/node
v1: digest: sha256:912df0a389b1479ce9799ed9873d846432743fcd91e282fc84cf38308a20adaa size: 1998
[ec2-user@ip-172-31-19-170 NodeJs_based_source]$
[ec2-user@ip-172-31-19-170 NodeJs_based_source]$ docker pull sunith18/node-app:v1
v1: Pulling from sunith18/node-app
Digest: sha256:912df0a389b1479ce9799ed9873d846432743fcd91e282fc84cf38308a20adaa
Status: Image is up to date for sunith18/node-app:v1
docker.io/sunith18/node-app:v1
[ec2-user@ip-172-31-19-170 NodeJs_based_source]$ docker images
REPOSITORY                                                TAG       IMAGE ID       CREATED          SIZE
node-app                                                  v1        8c8e3d3475f6   7 minutes ago    539MB
sunith18/node-app                                         v1        8c8e3d3475f6   7 minutes ago    539MB
java-app                                                  v1        f92afcf9ff49   32 minutes ago   323MB
sunith18/java-app                                         v1        f92afcf9ff49   32 minutes ago   323MB
sunith18/python-app                                       v1        9655337acb3d   50 minutes ago   141MB
frontend-app                                              v1        7830ea1b2492   4 hours ago      69.1MB
sunith18/frontend-app                                     v1        7830ea1b2492   4 hours ago      69.1MB
567487919764.dkr.ecr.us-east-1.amazonaws.com/frontend-app  v1        7830ea1b2492   4 hours ago      69.1MB
<none>                                                    <none>    81a7aa00c0ed   4 hours ago      98.3MB
hello-world                                               latest    e2ac70e7319a   6 months ago     10.1kB
[ec2-user@ip-172-31-19-170 NodeJs_based_source]$
```



Conclusion:

The Node.js (React) application zomato_clone was successfully containerized using a node:20-alpine base image, built with tag v1, tested locally on port 3000, and pushed to Docker Hub as sunith18/node-app:v1.

- Write a **docker-compose file** to set up WordPress with a MySQL database.

Objective

Write a docker-compose.yml file that runs WordPress with a MySQL database as separate services with persistent storage.

Purpose

- Deploy a real-world multi-container application with a single command.
- Demonstrate Docker Compose orchestration of dependent services.
- Ensure data persistence across container restarts using Docker volumes.

- Understand service networking (WordPress connecting to MySQL over a bridge network).

Environment

- **Host OS:** Amazon Linux 2023 (EC2)
- **Docker Version:** 25.0.14
- **Compose Version:** v2.x (installed as Docker CLI plugin)
- **WordPress Image:** wordpress:latest
- **MySQL Image:** mysql:8.0
- **Access Port:** 80
- **Compose file path:** ~/wordpress/docker-compose.yml

Prerequisites

- Docker Engine installed and running.
- Docker Compose plugin installed.
- Port 80 open in EC2 Security Group (Custom TCP → 0.0.0.0/0).
- At least 1 GB free disk space (WordPress + MySQL images).

Architecture

```
EC2 Host (Amazon Linux 2023)
|
|-- docker-compose.yml
|
|-- Bridge network: wp-network
     |
     |-- Container: wp-site (wordpress:latest)
     |    |-- Port 80 (host) → 80 (container)
     |    |-- Volume: wp_data → /var/www/html
     |
     |-- Container: wp-db (mysql:8.0)
          |-- No host port exposed
          |-- Volume: db_data → /var/lib/mysql
          |
          |-- Env: MYSQL_DATABASE=wordpress
                 MYSQL_USER=wpuser
                 MYSQL_PASSWORD=wppass
                 MYSQL_ROOT_PASSWORD=rootpass

Browser → http://<EC2_PUBLIC_IP> → wp-site → wp-db
```

Steps Performed

Step 1 — Install Docker Compose plugin (Amazon Linux 2023)

Since docker-compose-plugin is not available in AL2023 repos, we installed the binary directly:

```
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-
  x86_64 \
 -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
docker compose version
```

Step 2 — Create the project folder

mkdir -p ~/wordpress && cd ~/wordpress

Step 3 — Write the compose file

nano docker-compose.yml

Contents:

```
version: '3.8'

services:
  db:
    image: mysql:8.0
    container_name: wp-db
    restart: always
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
      MYSQL_ROOT_PASSWORD: rootpass
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - wp-network

  wordpress:
    image: wordpress:latest
    container_name: wp-site
    restart: always
    ports:
      - "80:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html
    depends_on:
      - db
    networks:
      - wp-network

volumes:
  db_data:
  wp_data:

networks:
  wp-network:
    driver: bridge
```

Step 4 — Start the stack

docker compose up -d

Step 5 — Verify containers

docker compose ps

Output (expected):

NAME IMAGE STATUS PORTS
wp-db mysql:8.0 Up (healthy) 3306/tcp
wp-site wordpress:latest Up 0.0.0.0:80->80/tcp

Step 6 — Check logs for successful initialization

docker compose logs --tail=30

Step 7 — Access WordPress in the browser

Opened: http://<EC2_PUBLIC_IP>

Completed the WordPress setup wizard:

- Selected language
- Entered site title, admin username, password, email
- Clicked **Install WordPress**

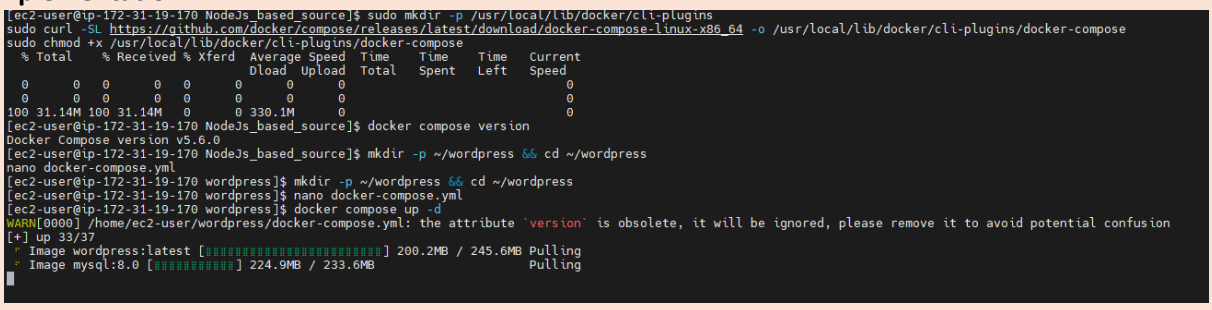
Step 8 — Verify data persistence

docker compose down
docker compose up -d

Output / Result

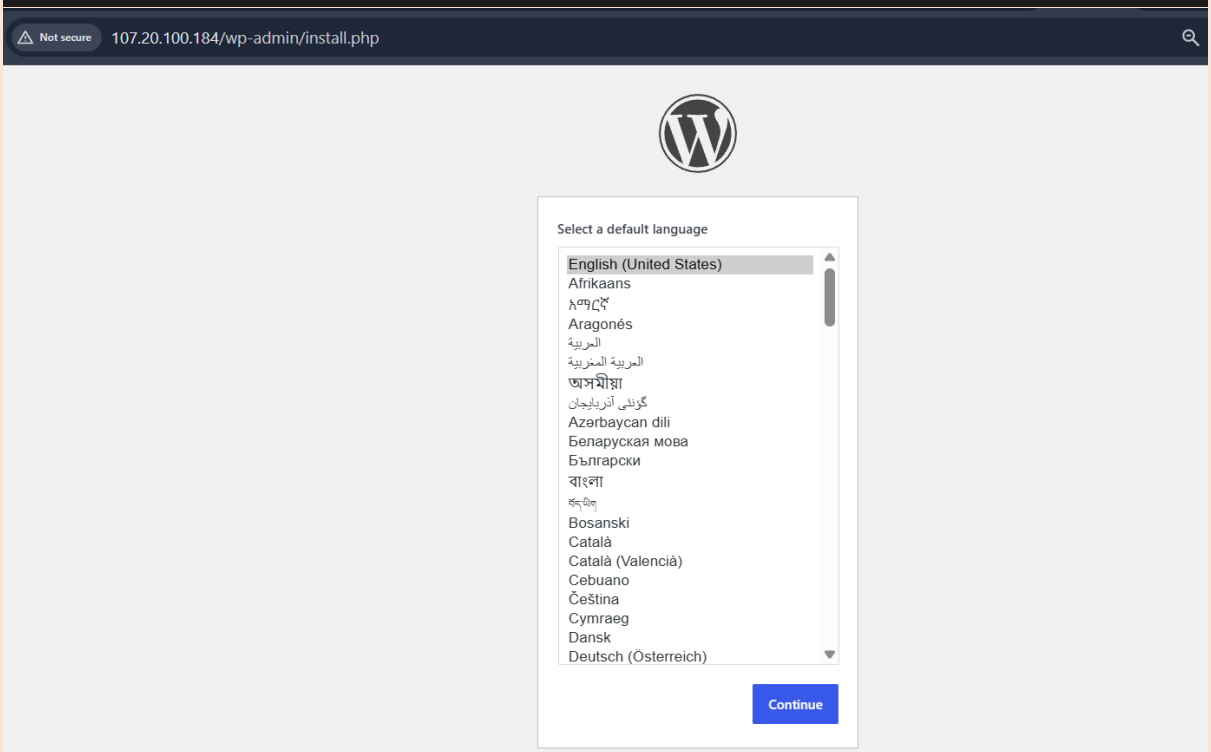
- Docker Compose v2.x installed successfully on Amazon Linux 2023.
- Two containers running: wp-site (WordPress) and wp-db (MySQL 8.0).
- WordPress accessible in the browser at http://<EC2_PUBLIC_IP>.
- Setup wizard completed; site is live.
- Data persisted across docker compose down / up cycles via named volumes.

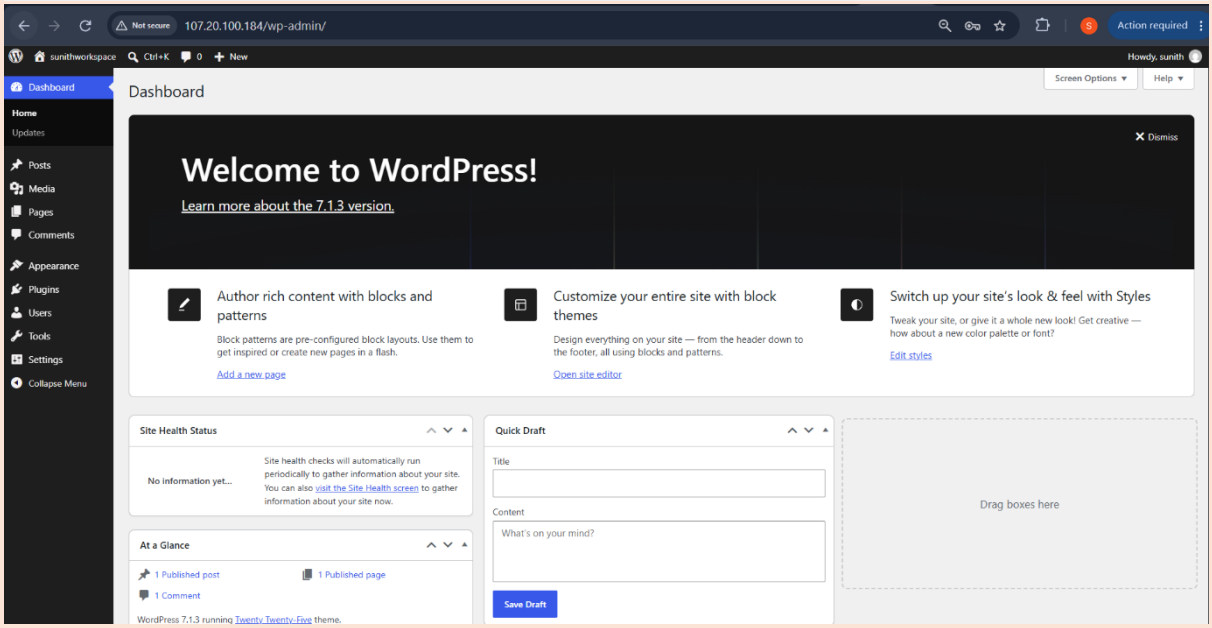
Implementation:



Docker-Challenge

```
[ec2-user@ip-172-31-19-170 NodeJs_based_source]$ sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0                              0
  0     0    0     0    0     0      0      0                              0
100 31.14M 100 31.14M    0     0 330.1M      0                              0
[ec2-user@ip-172-31-19-170 NodeJs_based_source]$ docker compose version
Docker Compose version v5.6.0
[ec2-user@ip-172-31-19-170 NodeJs_based_source]$ mkdir -p ~/wordpress && cd ~/wordpress
nano docker-compose.yml
[ec2-user@ip-172-31-19-170 wordpress]$ mkdir -p ~/wordpress && cd ~/wordpress
[ec2-user@ip-172-31-19-170 wordpress]$ nano docker-compose.yml
[ec2-user@ip-172-31-19-170 wordpress]$ docker compose up -d
WARN[0000] /home/ec2-user/wordpress/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 42/42
 ✔ Image wordpress:latest        Pulled
 ✔ Image mysql:8.0               Pulled
 ✔ Volume wordpress_db_data      Created
 ✔ Volume wordpress_wp_data      Created
 ✔ Network wordpress_wp-network  Created
 ✔ Container wp-db               Started
 ✔ Container wp-site             Started
[ec2-user@ip-172-31-19-170 wordpress]$
```





Conclusion

WordPress and MySQL were successfully deployed as a two-container stack using Docker Compose on Amazon Linux 2023. The stack demonstrates service orchestration (WordPress depends on MySQL), inter-service networking (via the wp-network bridge), and data persistence (via named volumes db_data and wp_data). This concludes the multi-container task and completes the full assignment.